

SISTEMAS DE INTELIGENCIA ARTIFICIAL · TRABAJO PRÁCTICO 2

Algoritmos genéticos

Ejercicio 1 · Representación de imágenes mediante caracteres ASCII

Ejercicio 2 · Aproximación de imágenes mediante triángulos

Facundo Lasserre · Gianna Lucía Nacuzzi · Javier Bautista Peral Belmont · Manuel José Santamarina Balbín · Santiago Mesa Rubio

EJERCICIO 1

Representación de imágenes mediante caracteres ASCII

Ejercicio de análisis: cómo se plantearía el problema con un algoritmo genético

EJERCICIO 1

Aproximación de imágenes con ASCII

Entrada

- Imagen objetivo
- Grilla de $N \times N$ celdas
- Alfabeto ASCII y parámetros del AG

Salida

- Mejor mapa ASCII
- Mapa de caracteres y fitness

Objetivo

Encontrar un mapa de caracteres que se parezca lo más posible a la imagen objetivo.

Cada individuo representa una imagen ASCII completa y el algoritmo mejora sucesivamente la aproximación.



EJERCICIO 1

Representación genética

I

Individuo

Mapa ASCII completo de $N \times N$

G

Genotipo

Matriz ordenada de N^2 genes

g

Gen y alelo

Una celda de la grilla y el carácter que contiene

AsciiCell

```
locus = (fila, columna)
alelo = carácter ∈ alfabeto permitido
fenotipo = imagen renderizada
```

.	.	:	:	.	.
.	#	@	@	#	.
:	@	@	@	@	:
.	#	@	@	#	.
.	.	:	:	.	.

Gen en locus (2, 2), alelo "@"

Reglas

- Cada celda contiene un carácter permitido
- La matriz conserva tamaño $N \times N$
- Fuente, tamaño y espaciado quedan fijos
- El mismo genotipo produce el mismo fenotipo

EJERCICIO 1

Evolución de la imagen ASCII



El ciclo continúa hasta alcanzar el máximo de generaciones, una semejanza suficiente o estancamiento.

El algoritmo conserva el mejor mapa ASCII encontrado durante la ejecución.

La selección de padres puede basarse en el fitness; la cruce conserva cada posición y la mutación cambia un carácter válido.

EJERCICIO 2

Aproximación de imágenes mediante triángulos

Implementación, experimentación y resultados

EL PROBLEMA

Objetivo, entradas y salidas

Entrada

- Imagen objetivo
- Cantidad fija de triángulos
- Hiperparámetros del AG

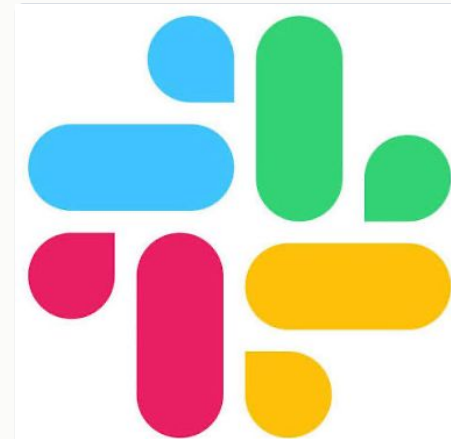
Salida

- Mejor imagen encontrada
- Lista de triángulos y métricas

Objetivo

Encontrar una secuencia ordenada de triángulos que minimice la diferencia respecto de la imagen objetivo.

El algoritmo optimiza posiciones, colores, transparencias y orden de dibujo.



Las tres imágenes objetivo, de menor a mayor dificultad

EL PROBLEMA

Preguntas de experimentación

¿Qué combinación produce mejores aproximaciones bajo el mismo presupuesto?

CAPACIDAD

¿Cómo influyen la resolución y la cantidad de triángulos?

OPERADORES

¿Qué métodos de selección, cruza, mutación y supervivencia funcionan mejor manteniendo fijo el resto?

ROBUSTEZ

¿Las decisiones se sostienen con distintas semillas?

No asumimos que un método fuera universalmente mejor: diseñamos campañas para decidir.

Representación genética

I

Individuo

Solución candidata completa

C

Cromosoma

Secuencia ordenada de T genes

G

Gen

Un triángulo completo

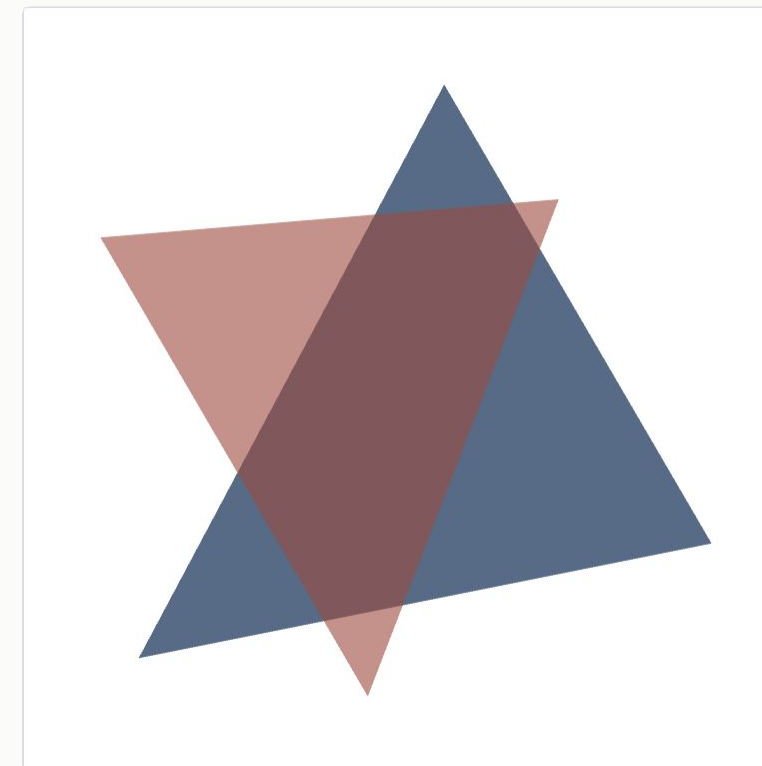
TriangleGene

$v_1 = (x_1, y_1)$

$v_2 = (x_2, y_2)$

$v_3 = (x_3, y_3)$

$color = (R, G, B, A)$



Validez

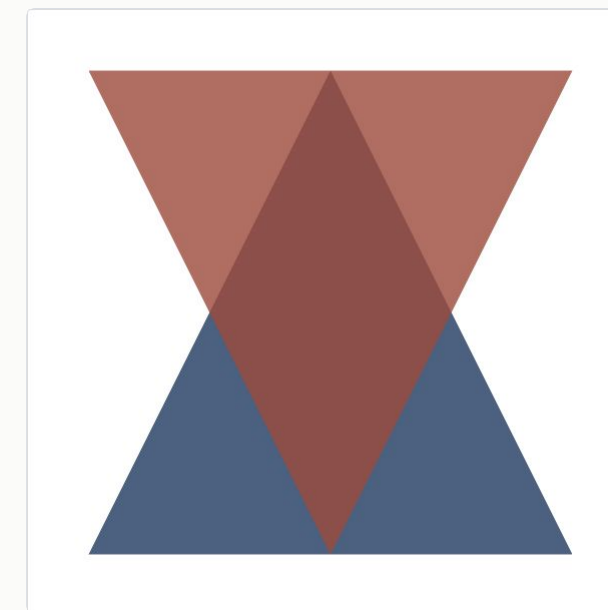
- Vértices dentro del canvas, coordenadas en $[0, 1]$
- Área positiva
- RGBA válido y $\alpha > 0$

DISEÑO

Transparencia y composición de capas

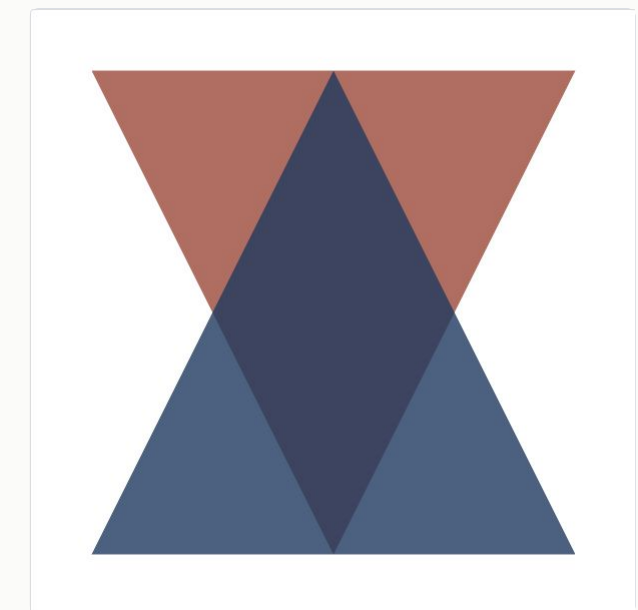
A controla la opacidad. Aunque ambos colores contribuyen, el triángulo dibujado último tiene mayor peso en la mezcla; por eso invertir el orden cambia el color resultante.

El renderer recorre el cromosoma de izquierda a derecha y compone las capas sobre el canvas.



A, luego B

≠



B, luego A

Cambiar el orden puede cambiar el fenotipo aunque los triángulos sean los mismos: el orden de los genes forma parte de la solución.

Función de error y fitness

```
NMSE = promedio((objetivo - generado)²) / 255²
```

```
fitness = max(ε, 1 - NMSE)
```

NMSE = 0

imágenes idénticas

fitness ↑

mejor individuo

- Mide el error por píxel, está acotado y permite comparar corridas de manera reproducible.
- El motor maximiza fitness y reporta el error interpretable.
- Puede asignar errores similares a imágenes que una persona percibe como visualmente diferentes.

Adoptamos NMSE como función de error y contrastamos siempre los números con las imágenes objetivo.

MOTOR

Elección de cruza y mutación

CRUZA

Cómo combinar dos cromosomas

Un punto

Conserva segmentos consecutivos del cromosoma.

Uniforme

Combina triángulos distribuidos entre ambos padres.

POR QUÉ LAS COMPARAMOS

Conservación de bloques frente a mezcla distribuida.

MUTACIÓN

Cómo introducir variación

Un gen

Modifica exactamente un triángulo.

Multigénica

Puede modificar cero, uno o varios triángulos.

POR QUÉ LAS COMPARAMOS

Cambio puntual garantizado frente a exploración distribuida.

Dos comportamientos contrastantes, evaluados después bajo el mismo presupuesto.

Configuración de las corridas

config.json

Qué puede cambiarse sin tocar el código

ENTRADA	input.image input.working_max_side
CAPACIDAD	representation.triangle_count genetic.population_size / offspring_count
OPERADORES	genetic.parent_selection / crossover genetic.mutation / survival
EJECUCIÓN	termination run.seed

Criterios de corte

Termina con el primero que se cumpla

- Máximo de generaciones
- NMSE objetivo
- Estancamiento
- Tiempo máximo

Archivos por corrida

Configuración efectiva, evolución y mejor solución

- config.effective.json
- metrics.csv
- best.png
- triangles.json
- metadata.json

La semilla y la configuración efectiva hacen reproducible cada corrida.

Diseño experimental

5

semillas por condición

=

mismo presupuesto dentro de
cada campaña

1

factor principal modificado por
vez

3

métricas: NMSE, AUC y
diversidad

Comparación entre imágenes

Ordenamos las condiciones dentro de cada objetivo. No promediamos NMSE absolutos de imágenes con distinta dificultad.

Robustez

Repetimos cada condición con cinco semillas para que la decisión no dependiera de una única corrida.

EXPERIMENTACIÓN

Etapas de la experimentación

1

Resolución

2

Triángulos

3

Selección

4

Cruza

5

Mutación

6

Supervivencia

7

Comprobación
final

8

Corridas
extendidas

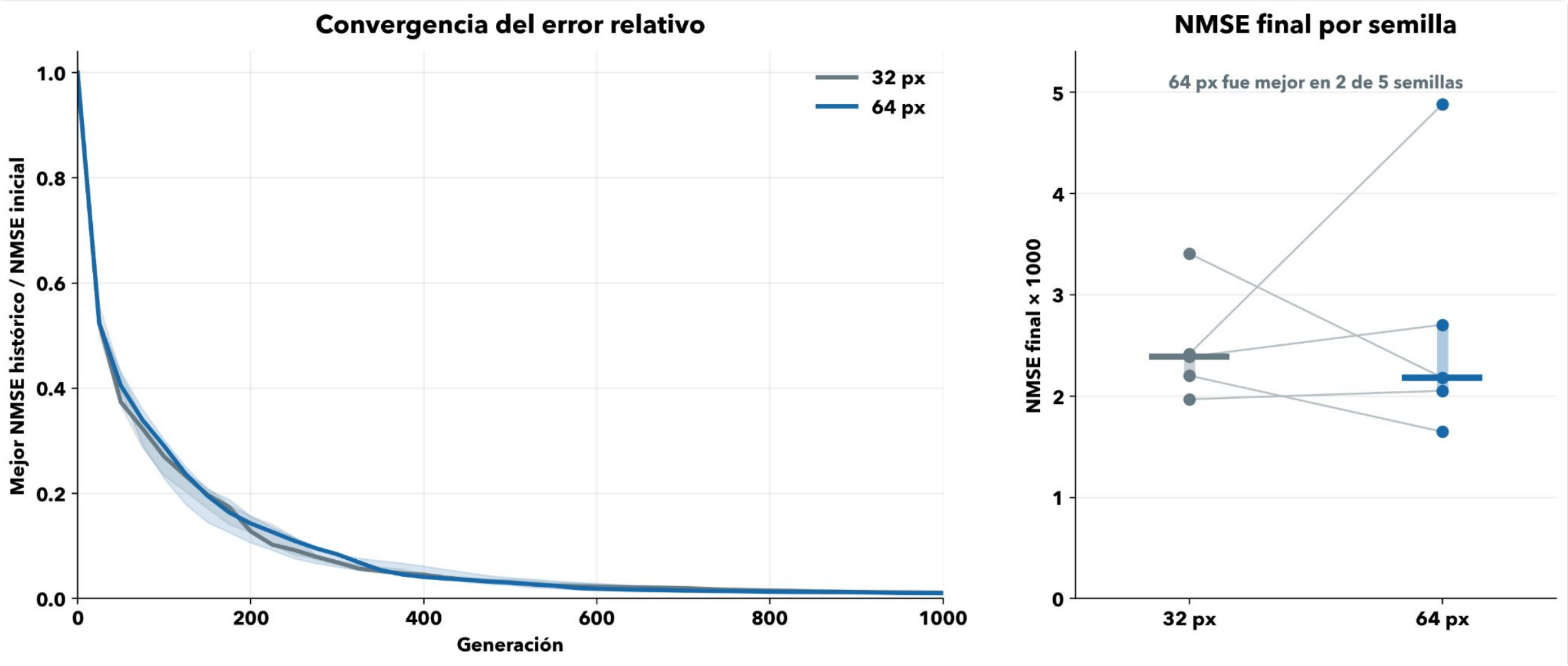
163

corridas completas

Cada etapa redujo las alternativas para la siguiente. En selección mantuvimos dos candidatos por sus resultados cercanos.

El diseño secuencial encuentra una configuración razonable, no garantiza el óptimo global.

Efecto de la resolución



8,6 %

menor NMSE mediano con 64 px

2 de 5

semillas donde ganó 64 px

11,90 s → 14,48 s

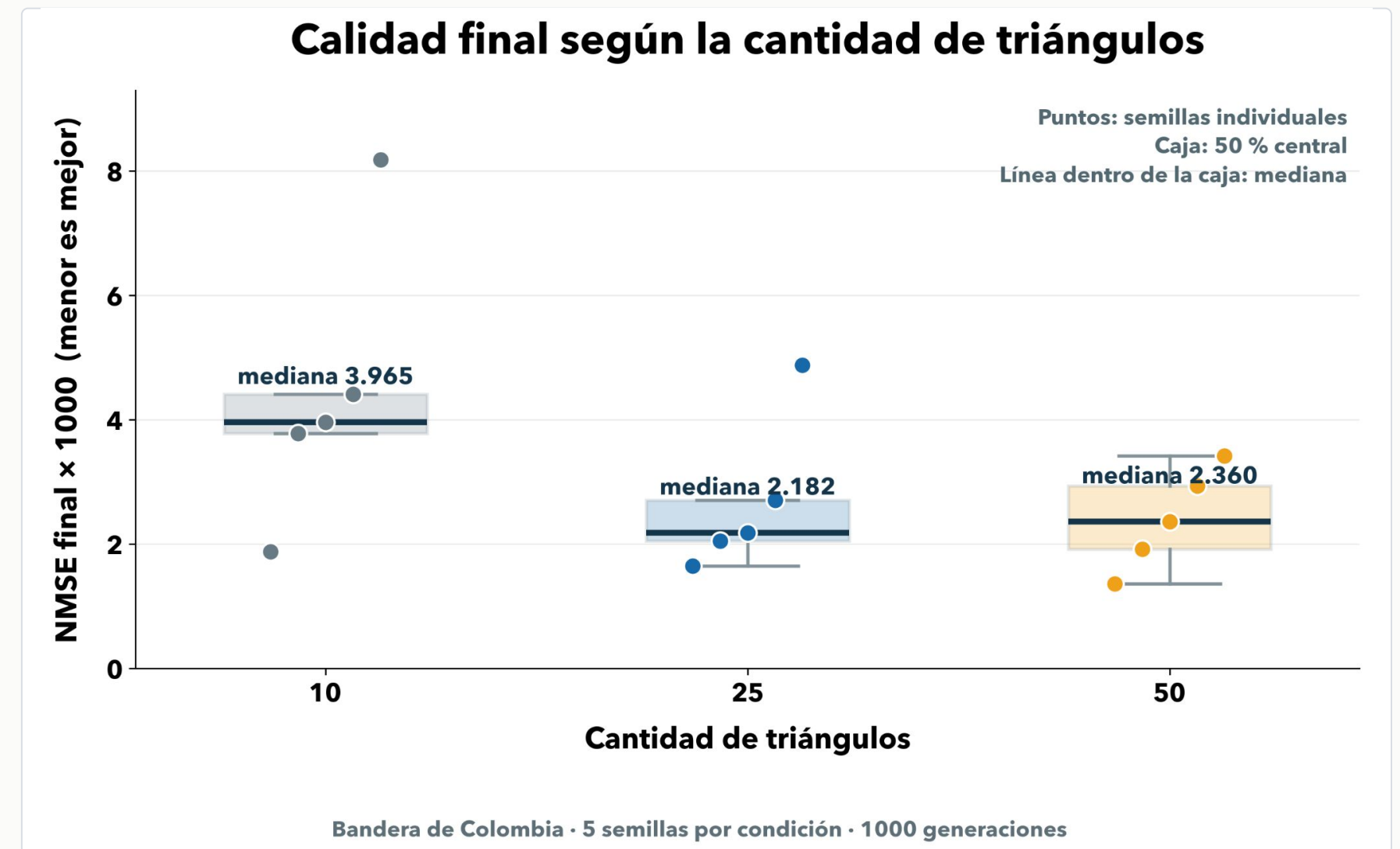
tiempo por corrida

Los rangos centrales se superponen. Elegimos 64 px por el detalle adicional y un costo aceptable, con evidencia numérica moderada.

Efecto de la cantidad de triángulos

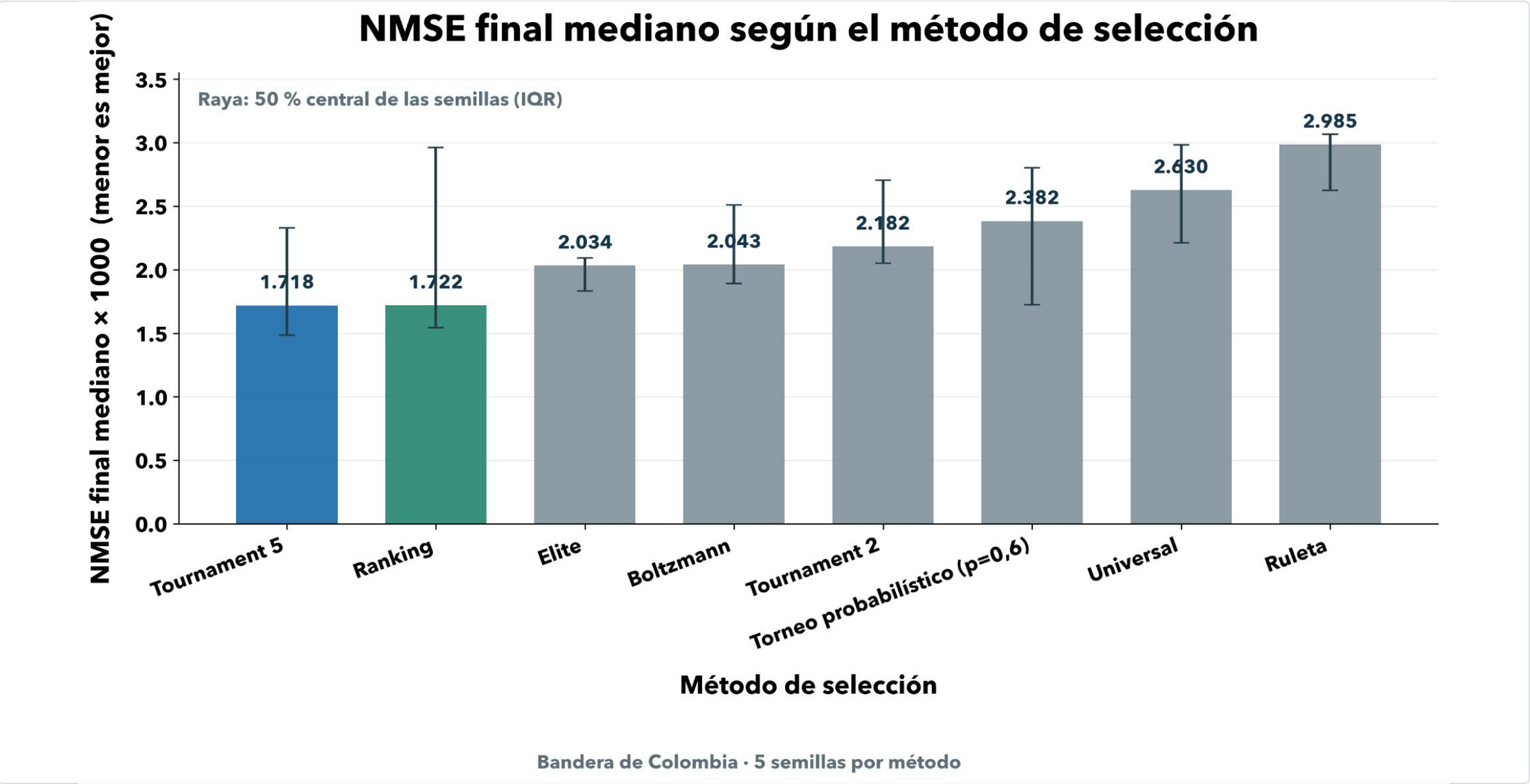


10 no alcanzaron. 50 casi duplicaron el tiempo de 25 y no aprovecharon su mayor capacidad en 1000 generaciones.



Más capacidad también amplía el espacio de búsqueda y no garantiza una mejora bajo el mismo presupuesto.

Comparación de los métodos de selección



FINALISTAS

Tournament 5

Mayor velocidad Menor diversidad

Ranking

Resultado similar Mayor diversidad

La decisión surgió de comparar todos los selectores solicitados, no de elegir dos arbitrariamente.

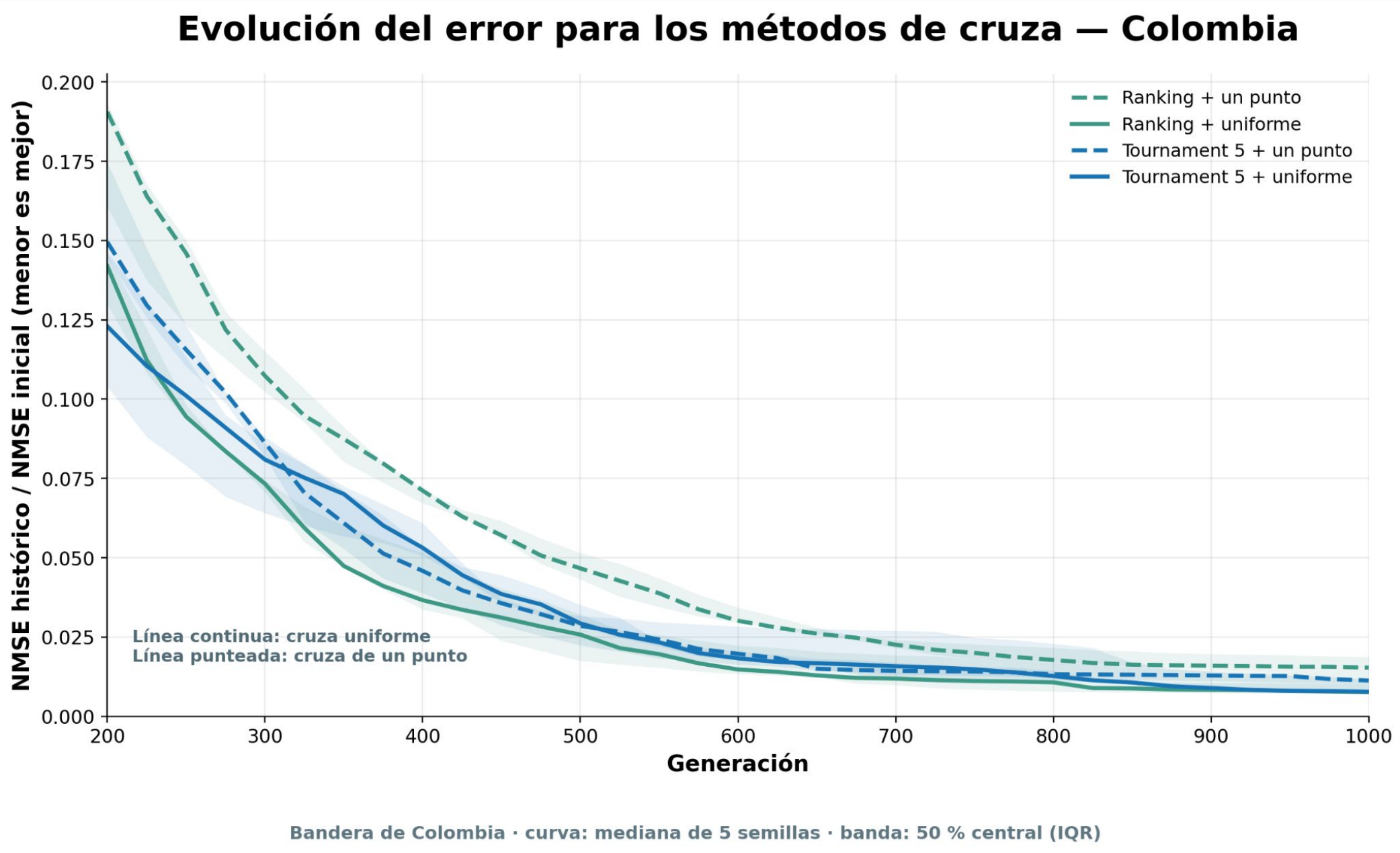
Comparación entre Tournament 5 y Ranking

Tournament 5			Ranking	
	MÁS RÁPIDO			MÁS DIVERSO
0,001718	0,0975	≈	0,001722	0,1011
NMSE mediano	AUC normalizada		NMSE mediano	AUC normalizada
0,000830	2 de 5	0,23%	0,001593	3 de 5
Diversidad final	Semillas ganadas	de diferencia en NMSE	Diversidad final	Semillas ganadas

Empate práctico: ambos selectores avanzaron a la campaña de cruza.

AUC menor = mejora más rápida

Cruza e interacción con la selección



Tournament 5

Uniforme: 0,001718

Un punto: 0,002314

Mejora: 26 %

Ranking

Uniforme: 0,001722

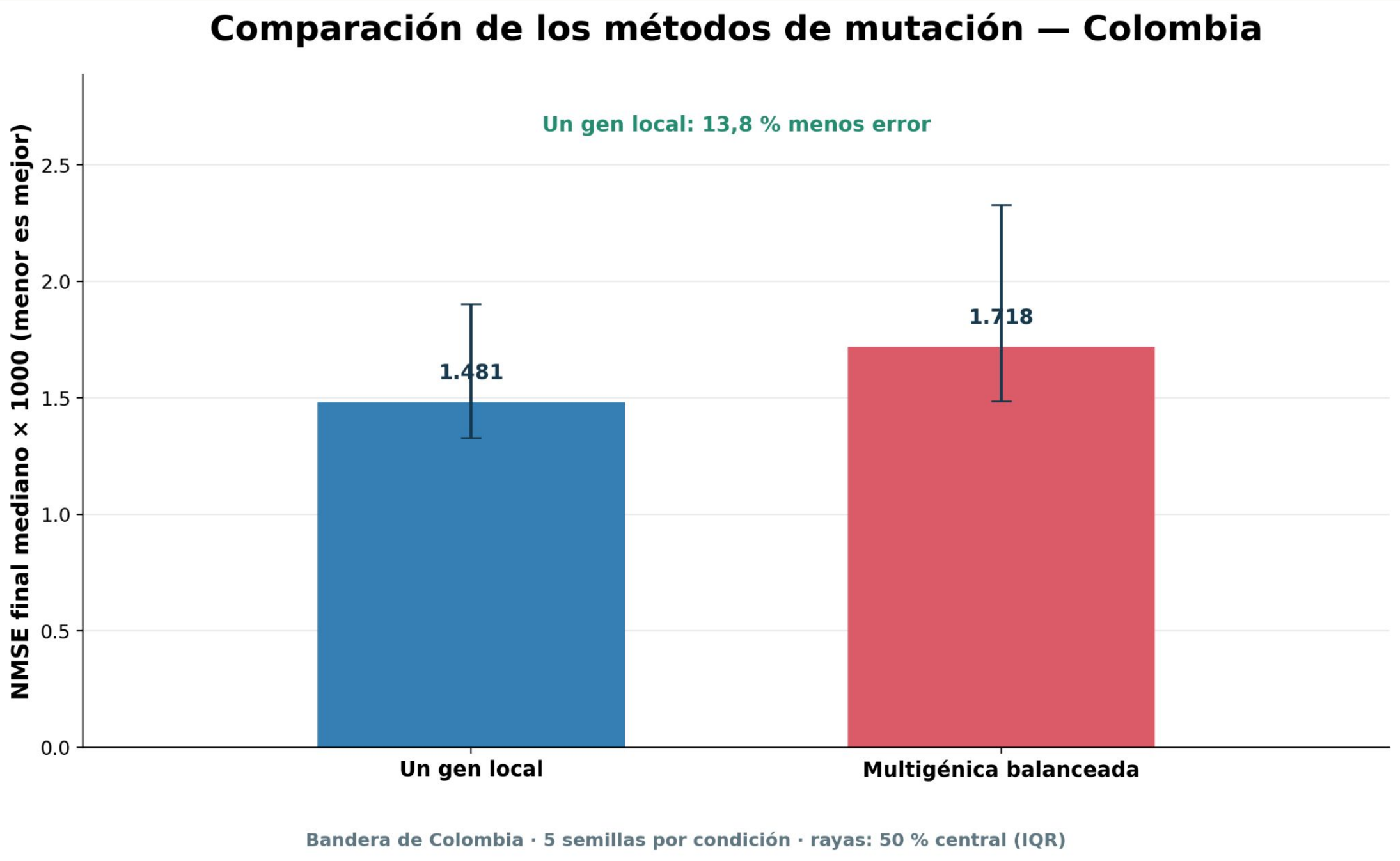
Un punto: 0,003073

Mejora: 44 %

La crusa uniforme ganó con ambos selectores; combinada con Tournament 5 obtuvo el menor NMSE.

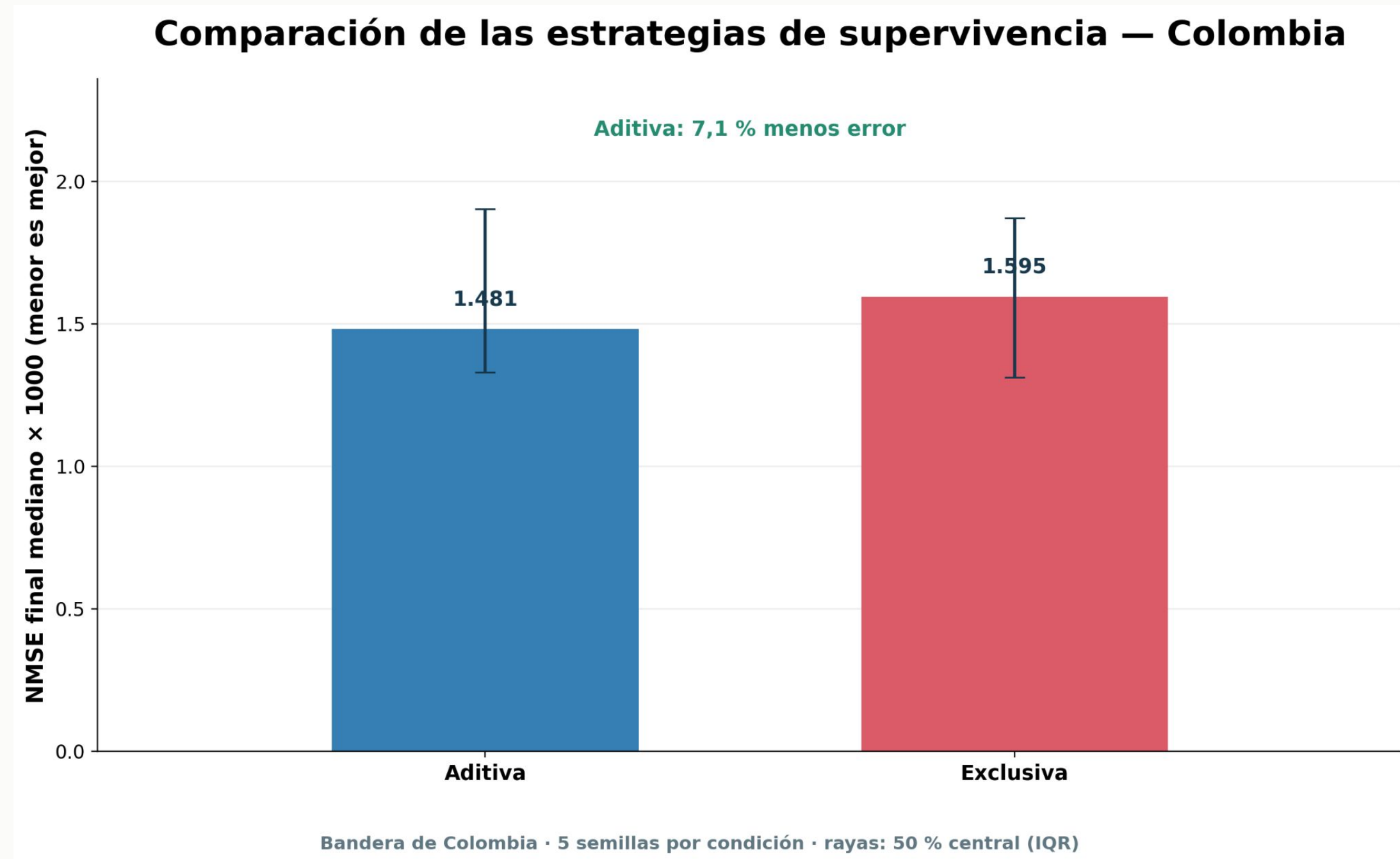
Como los genes consecutivos no representan necesariamente una misma región de la imagen, la crusa uniforme puede combinar triángulos útiles distribuidos por todo el cromosoma.

Comparación de métodos de mutación



Con $p = 1/T$, la multigénica puede producir hijos sin ningún cambio; single siempre modifica uno.

Comparación de estrategias de supervivencia



Elegimos aditiva, aunque la diferencia final fue pequeña.

RESULTADO

Resultados con la configuración final

COLOMBIA



objetivo



resultado

$25 \text{ triángulos} \cdot 1000 \text{ gen NMSE} \times 10^3 = 1,481$

SEÑAL



objetivo



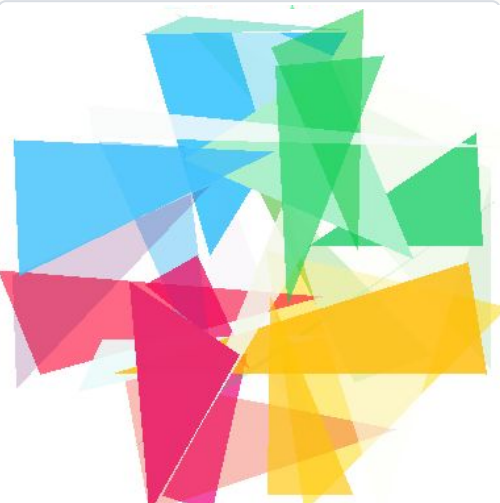
resultado

$25 \text{ triángulos} \cdot 1000 \text{ gen NMSE} \times 10^3 = 13,635$

ÍCONO



objetivo



resultado

$25 \text{ triángulos} \cdot 1000 \text{ gen NMSE} \times 10^3 = 18,887$

resolución de trabajo 64 px

población 50

Tournament 5

cruza uniforme

mutación de un gen

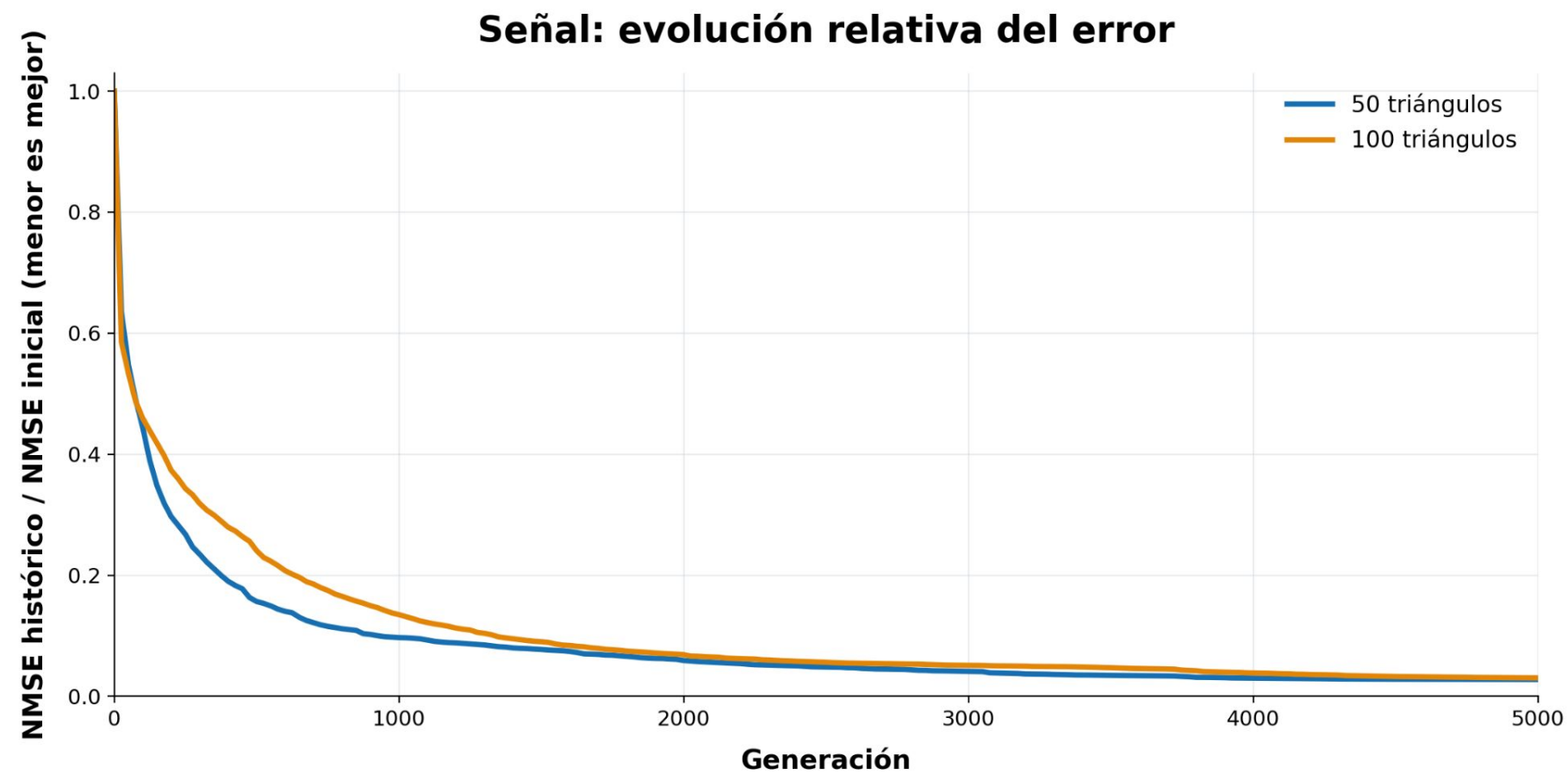
supervivencia aditiva con Elite

Las comparaciones usaron 1000 generaciones.

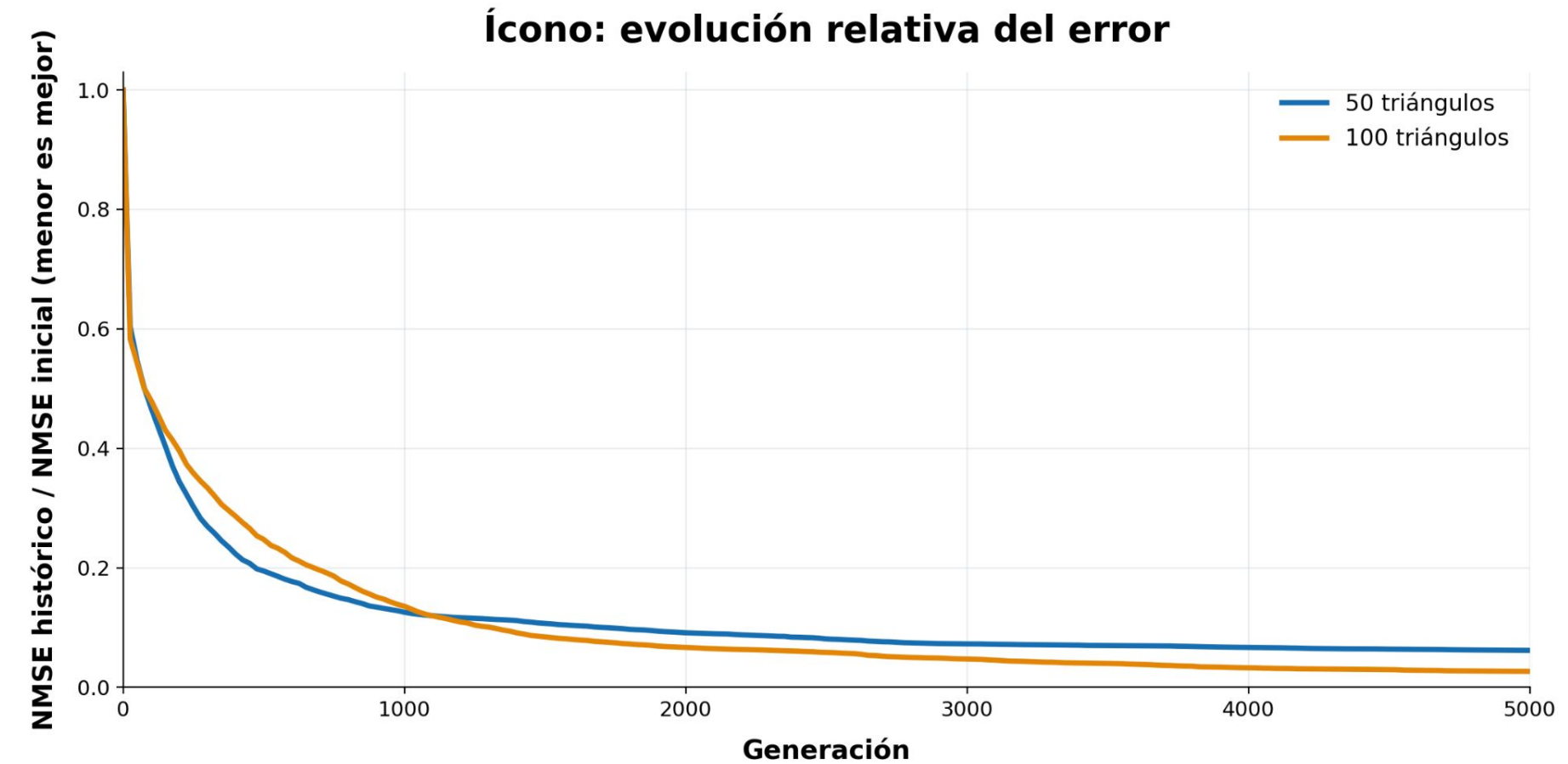
RESULTADO

Evolución según la cantidad de triángulos

SEÑAL



ÍCONO



Al extender el presupuesto, 100 triángulos superaron a 50 en el ícono. En la señal, 50 conservaron el menor error.

RESULTADO

Resultados de las corridas extendidas

SEÑAL



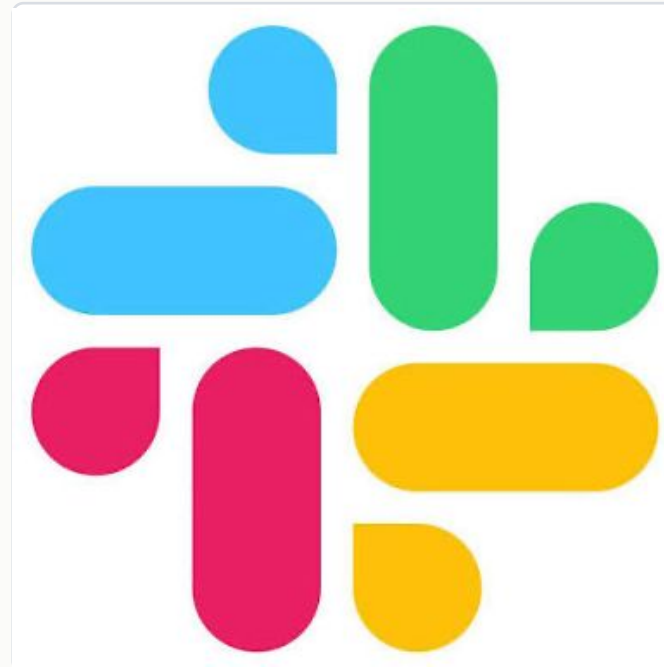
objetivo



resultado

50 triángulos · 5000 gen $\text{NMSE} \times 10^3 = 3,633$

ÍCONO



objetivo



resultado

100 triángulos · 5000 gen $\text{NMSE} \times 10^3 = 3,521$

resolución de trabajo 64 px

población 50

Tournament 5

cruza uniforme

mutación de un gen

supervivencia aditiva con Elite

Conclusiones

- 1** La capacidad útil dependió del objetivo y del presupuesto: con 1000 generaciones, 25 triángulos rindieron mejor en Colombia. Con 5000, 100 superaron a 50 en el ícono, pero no en la señal.
- 2** La cruza uniforme mostró la ventaja más consistente: redujo el NMSE frente a un punto un 26 % con Tournament 5 y un 44 % con Ranking en Colombia.
- 3** La mutación de un gen obtuvo menor error: su NMSE mediano fue 13,8 % menor que el de la multigénica en Colombia.
- 4** Selección y supervivencia mostraron diferencias más pequeñas: Tournament 5 aventajó a Ranking por solo 0,23 %, mientras que la supervivencia aditiva redujo la mediana 7,1 % con IQR superpuestos.

El diseño experimental mejoró las aproximaciones y abrió nuevas líneas para seguir optimizando el algoritmo.